

5 Computation:

5.1 discrete state-space

```
! 1. grid search

PROGRAM growth
IMPLICIT NONE
INTEGER, PARAMETER:: n=100
REAL*8, PARAMETER:: beta=0.95D+00,alpha=0.36D+00,A=4.0D+00
REAL*8:: k(n),v(n),Tv(n),v_true(n),k_ss,kmin,kmax,policy_true(n),&
policy1(n)=0.D0,temp(n),consumption,update
INTEGER:: i,j,tempindex(1),iters

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
! compute the true value function
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
k_ss=(A*beta*alpha)**(1/(1-alpha))
kmin=0.1*k_ss
kmax=4*k_ss
DO i=1,n
    k(i)=kmin+(i-1)*(kmax-kmin)/(n-1)
    v_true(i)=(DLOG(A*(1-beta*alpha))+alpha*beta/(1-alpha*beta)&
    *DLOG(A*alpha*beta))/(1-beta)+alpha/(1-beta*alpha)*DLOG(k(i))
    policy_true(i)=alpha*beta*A*k(i)**alpha
ENDDO
```

```

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!Method 1: start Value function iteration by grid search
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

v=0.0D0
update=1.0D+00
iters=1

DO WHILE (update>1.D-6)
DO i=1,n
DO j=1,n
consumption=A*k(i)**alpha-k(j)
IF (consumption>0) THEN
temp(j)=DLOG(consumption)+beta*v(j)
ELSE
temp(j)=-1.0D+10
ENDIF
ENDDO
tempindex=MAXLOC(temp)
policy1(i)=k(tempindex(1))
Tv(i)=temp(tempindex(1))
ENDDO
update=MAXVAL(ABS(v-Tv))
print *, 'iters:', iters, ' update=', update
v=Tv
iters=iters+1
ENDDO

```

```

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

! report results

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

print *, 'Grid Search, value function error:', MAXVAL(ABS(v-v_true))
print *, 'Grid Search, policy function error:', MAXVAL(ABS(policy1-policy_true))


!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

! save results

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

      OPEN(11,file='growth.csv')
      WRITE(11,"(16(A20,', '))") 'k', 'value_true', 'value', &
      'policy_true', 'policy1', 'policy1-policy_true'
      DO i=1,n
      10      format(10(f18.10,', '))
              write(11,10) k(i),v_true(i),v(i),&
              policy_true(i),policy1(i),policy1(i)-policy_true(i)
      ENDDO
      CLOSE(11)
      END PROGRAM growth

```

5.2 optimization

To solve $\max_x f(x)$, we could solve the root-finding problem $f'(x) = 0$ when f is differentiable. If f is not differentiable, or f' is difficult to compute numerically, we need other methods.

Consider a function define on $[0, 1]$, and pick two points $x_1, x_2 \in (0, 1)$ such that

1. $x_1 < x_2$ is symmetric around 0.5.
2. In $[0, x_2]$, x_1 is the second point. That is $x_1 : x_2 = x_2 : 1$.

This gives $x_2^2 = 1 - x_2$, or $x_2 = \frac{\sqrt{5}-1}{2} \approx 0.618$, the golden ratio.

```
SUBROUTINE golden(ax,cx,tol,xmax,fmax)
```

```
REAL*8 ax,cx,tol,xmax,fmax,R,C,k
```

```
PARAMETER (R=.61803399,C=1.-R)
```

```
REAL*8 f1,f2,x0,x1,x2,x3
```

```
x0=ax
```

```
x3=cx
```

```
x1=R*x0+C*x3
```

```
x2=R*x1+C*x3
```

```
f1=f(x1)
```

```
f2=f(x2)
```

```
DO WHILE (abs(x3-x0).gt.tol)
```

```
    if(f2.gt.f1)then
```

```
        x0=x1
```

```
        x1=x2
```

```
        x2=R*x1+C*x3
```

```

        f1=f2
        f2=f(x2)
    else
        x3=x2
        x2=x1
        x1=R*x2+C*x0
        f2=f1
        f1=f(x1)
    endif
ENDDO

if(f1.gt.f2)then
    fmax=f1
    xmax=x1
else
    fmax=f2
    xmax=x2
endif
END SUBROUTINE golden

```

```

w=c(i+1)-d(i)
h=xa(i+m)-x          h will never be zero, since this was tested in the ini-
t=(xa(i)-x)*d(i)/h    tializing loop.
dd=t-c(i+1)
if(dd.eq.0.)pause 'failure in ratint'
  This error condition indicates that the interpolating function has a pole at the re-
  quested value of x.
dd=w/dd
d(i)=c(i+1)*dd
c(i)=t*dd
enddo 12
if (2*ns.lt.n-m)then
  dy=c(ns+1)
else
  dy=d(ns)
  ns=ns-1
endif
y=y+dy
enddo 13
return
END

```

CITED REFERENCES AND FURTHER READING:

- Stoer, J., and Bulirsch, R. 1980, *Introduction to Numerical Analysis* (New York: Springer-Verlag), §2.2. [1]
- Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall), §6.2.
- Cuyt, A., and Wuytack, L. 1987, *Nonlinear Methods in Numerical Analysis* (Amsterdam: North-Holland), Chapter 3.

3.3 Cubic Spline Interpolation

Given a tabulated function $y_i = y(x_i)$, $i = 1 \dots N$, focus attention on one particular interval, between x_j and x_{j+1} . Linear interpolation in that interval gives the interpolation formula

$$y = Ay_j + By_{j+1} \quad (3.3.1)$$

where

$$A \equiv \frac{x_{j+1} - x}{x_{j+1} - x_j} \quad B \equiv 1 - A = \frac{x - x_j}{x_{j+1} - x_j} \quad (3.3.2)$$

Equations (3.3.1) and (3.3.2) are a special case of the general Lagrange interpolation formula (3.1.1).

Since it is (piecewise) linear, equation (3.3.1) has zero second derivative in the interior of each interval, and an undefined, or infinite, second derivative at the abscissas x_j . The goal of cubic spline interpolation is to get an interpolation formula that is smooth in the first derivative, and continuous in the second derivative, both within an interval and at its boundaries.

Suppose, contrary to fact, that in addition to the tabulated values of y_i , we also have tabulated values for the function's second derivatives, y'' , that is, a set

Sample page from NUMERICAL RECIPES IN FORTRAN 77: THE ART OF SCIENTIFIC COMPUTING (ISBN 0-521-43064-X)
 Copyright (C) 1986-1992 by Cambridge University Press. Programs Copyright (C) 1986-1992 by Numerical Recipes Software.
 Permission is granted for internet users to make one paper copy for their own personal use. Further reproduction, or any copying of machine-readable files (including this one), to any server computer, is strictly prohibited. To order Numerical Recipes books, diskettes, or CDROMs visit website <http://www.nr.com> or call 1-800-872-7423 (North America only), or send email to trade@cup.cam.ac.uk (outside North America).

of numbers y''_i . Then, within each interval, we can add to the right-hand side of equation (3.3.1) a cubic polynomial whose second derivative varies linearly from a value y''_j on the left to a value y''_{j+1} on the right. Doing so, we will have the desired continuous second derivative. If we also construct the cubic polynomial to have zero values at x_j and x_{j+1} , then adding it in will not spoil the agreement with the tabulated functional values y_j and y_{j+1} at the endpoints x_j and x_{j+1} .

A little side calculation shows that there is only one way to arrange this construction, namely replacing (3.3.1) by

$$y = Ay_j + By_{j+1} + Cy''_j + Dy''_{j+1} \quad (3.3.3)$$

where A and B are defined in (3.3.2) and

$$C \equiv \frac{1}{6}(A^3 - A)(x_{j+1} - x_j)^2 \quad D \equiv \frac{1}{6}(B^3 - B)(x_{j+1} - x_j)^2 \quad (3.3.4)$$

Notice that the dependence on the independent variable x in equations (3.3.3) and (3.3.4) is entirely through the linear x -dependence of A and B , and (through A and B) the cubic x -dependence of C and D .

We can readily check that y'' is in fact the second derivative of the new interpolating polynomial. We take derivatives of equation (3.3.3) with respect to x , using the definitions of A, B, C, D to compute $dA/dx, dB/dx, dC/dx$, and dD/dx . The result is

$$\frac{dy}{dx} = \frac{y_{j+1} - y_j}{x_{j+1} - x_j} - \frac{3A^2 - 1}{6}(x_{j+1} - x_j)y''_j + \frac{3B^2 - 1}{6}(x_{j+1} - x_j)y''_{j+1} \quad (3.3.5)$$

for the first derivative, and

$$\frac{d^2y}{dx^2} = Ay''_j + By''_{j+1} \quad (3.3.6)$$

for the second derivative. Since $A = 1$ at x_j , $A = 0$ at x_{j+1} , while B is just the other way around, (3.3.6) shows that y'' is just the tabulated second derivative, and also that the second derivative will be continuous across (e.g.) the boundary between the two intervals (x_{j-1}, x_j) and (x_j, x_{j+1}) .

The only problem now is that we supposed the y''_i 's to be known, when, actually, they are not. However, we have not yet required that the *first* derivative, computed from equation (3.3.5), be continuous across the boundary between two intervals. The key idea of a cubic spline is to require this continuity and to use it to get equations for the second derivatives y''_i .

The required equations are obtained by setting equation (3.3.5) evaluated for $x = x_j$ in the interval (x_{j-1}, x_j) equal to the same equation evaluated for $x = x_j$ but in the interval (x_j, x_{j+1}) . With some rearrangement, this gives (for $j = 2, \dots, N-1$)

$$\frac{x_j - x_{j-1}}{6}y''_{j-1} + \frac{x_{j+1} - x_{j-1}}{3}y''_j + \frac{x_{j+1} - x_j}{6}y''_{j+1} = \frac{y_{j+1} - y_j}{x_{j+1} - x_j} - \frac{y_j - y_{j-1}}{x_j - x_{j-1}} \quad (3.3.7)$$

These are $N - 2$ linear equations in the N unknowns $y''_i, i = 1, \dots, N$. Therefore there is a two-parameter family of possible solutions.

For a unique solution, we need to specify two further conditions, typically taken as boundary conditions at x_1 and x_N . The most common ways of doing this are either

- set one or both of y_1'' and y_N'' equal to zero, giving the so-called *natural cubic spline*, which has zero second derivative on one or both of its boundaries, or
- set either of y_1'' and y_N'' to values calculated from equation (3.3.5) so as to make the first derivative of the interpolating function have a specified value on either or both boundaries.

One reason that cubic splines are especially practical is that the set of equations (3.3.7), along with the two additional boundary conditions, are not only linear, but also *tridiagonal*. Each y_j'' is coupled only to its nearest neighbors at $j \pm 1$. Therefore, the equations can be solved in $O(N)$ operations by the tridiagonal algorithm (§2.4). That algorithm is concise enough to build right into the spline calculational routine. This makes the routine not completely transparent as an implementation of (3.3.7), so we encourage you to study it carefully, comparing with `tridag` (§2.4).

```
SUBROUTINE spline(x,y,n,yp1,ypn,y2)
  INTEGER n,NMAX
  REAL yp1,ypn,x(n),y(n),y2(n)
  PARAMETER (NMAX=500)
```

Given arrays $x(1:n)$ and $y(1:n)$ containing a tabulated function, i.e., $y_i = f(x_i)$, with $x_1 < x_2 < \dots < x_N$, and given values $yp1$ and ypn for the first derivative of the interpolating function at points 1 and n , respectively, this routine returns an array $y2(1:n)$ of length n which contains the second derivatives of the interpolating function at the tabulated points x_i . If $yp1$ and/or ypn are equal to 1×10^{30} or larger, the routine is signaled to set the corresponding boundary condition for a natural spline, with zero second derivative on that boundary.

Parameter: $NMAX$ is the largest anticipated value of n .

```
  INTEGER i,k
  REAL p,qn,sig,un,u(NMAX)
  if (yp1.gt..99e30) then
    y2(1)=0.
    u(1)=0.
    *
  else
    y2(1)=-0.5
    u(1)=(3./(x(2)-x(1)))*((y(2)-y(1))/(x(2)-x(1))-yp1)
    endif
  do 11 i=2,n-1
    sig=(x(i)-x(i-1))/(x(i+1)-x(i-1))
    p=sig*y2(i-1)+2.
    y2(i)=(sig-1.)/p
    u(i)=(6.*((y(i+1)-y(i))/(x(i+1)-x(i))-(y(i)-y(i-1))
    / (x(i)-x(i-1)))/(x(i+1)-x(i-1))-sig*u(i-1))/p
    *
  enddo 11
  if (ypn.gt..99e30) then
    qn=0.
    un=0.
  else
    qn=0.5
    un=(3./(x(n)-x(n-1)))*(ypn-(y(n)-y(n-1))/(x(n)-x(n-1)))
    endif
  y2(n)=(un-qn*u(n-1))/(qn*y2(n-1)+1.)
  do 12 k=n-1,1,-1
    y2(k)=y2(k)*y2(k+1)+u(k)
  enddo 12
  return
END
```

The lower boundary condition is set either to be "natural" or else to have a specified first derivative.

This is the decomposition loop of the tridiagonal algorithm. $y2$ and u are used for temporary storage of the decomposed factors.

The upper boundary condition is set either to be "natural" or else to have a specified first derivative.

This is the backsubstitution loop of the tridiagonal algorithm.

It is important to understand that the program `spline` is called only *once* to process an entire tabulated function in arrays x_i and y_i . Once this has been done,

values of the interpolated function for any value of x are obtained by calls (as many as desired) to a separate routine `splint` (for “*spline interpolation*”):

```

SUBROUTINE splint(xa,ya,y2a,n,x,y)
  INTEGER n
  REAL x,y,xa(n),y2a(n),ya(n)
  Given the arrays xa(1:n) and ya(1:n) of length n, which tabulate a function (with the
  xa's in order), and given the array y2a(1:n), which is the output from spline above,
  and given a value of x, this routine returns a cubic-spline interpolated value y.
  INTEGER k,khi,klo
  REAL a,b,h
  klo=1
  khi=n
  We will find the right place in the table by means of bisection.
  This is optimal if sequential calls to this routine are at random
  values of x. If sequential calls are in order, and closely
  spaced, one would do better to store previous values of
  klo and khi and test if they remain appropriate on the
  next call.
1  if (khi-klo.gt.1) then
    k=(khi+klo)/2
    if (xa(k).gt.x) then
      khi=k
    else
      klo=k
    endif
  goto 1
endif
  klo and khi now bracket the input value of x.
h=xa(khi)-xa(klo)
if (h.eq.0.) pause 'bad xa input in splint' The xa's must be distinct.
a=(xa(khi)-x)/h
  Cubic spline polynomial is now evaluated.
b=(x-xa(klo))/h
y=a*ya(klo)+b*ya(khi)+
* ((a**3-a)*y2a(klo)+(b**3-b)*y2a(khi))*(h**2)/6.
  return
END

```

CITED REFERENCES AND FURTHER READING:

- De Boor, C. 1978, *A Practical Guide to Splines* (New York: Springer-Verlag).
- Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §§4.4–4.5.
- Stoer, J., and Bulirsch, R. 1980, *Introduction to Numerical Analysis* (New York: Springer-Verlag), §2.4.
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed. (New York: McGraw-Hill), §3.8.

3.4 How to Search an Ordered Table

Suppose that you have decided to use some particular interpolation scheme, such as fourth-order polynomial interpolation, to compute a function $f(x)$ from a set of tabulated x_i 's and f_i 's. Then you will need a fast way of finding your place in the table of x_i 's, given some particular value x at which the function evaluation is desired. This problem is not properly one of numerical analysis, but it occurs so often in practice that it would be negligent of us to ignore it.

Formally, the problem is this: Given an array of abscissas $xx(j)$, $j=1, 2, \dots, n$, with the elements either monotonically increasing or monotonically decreasing, and given a number x , find an integer j such that x lies between $xx(j)$ and $xx(j+1)$.

Sample page from NUMERICAL RECIPES IN FORTRAN 77: THE ART OF SCIENTIFIC COMPUTING (ISBN 0-521-43064-X)
 Copyright (C) 1986-1992 by Cambridge University Press. Programs Copyright (C) 1986-1992 by Numerical Recipes Software.
 Permission is granted for internet users to make one paper copy for their own personal use. Further reproduction, or any copying of machine-readable files (including this one), to any server computer, is strictly prohibited. To order Numerical Recipes books, diskettes, or CDROMs visit website <http://www.nr.com> or call 1-800-872-7423 (North America only), or send email to trade@cup.cam.ac.uk (outside North America).